Load by Design: Workload-Shaped Checkpoint Loading for Large Language Model Inference

Botir Khaltaev
Independent, United Kingdom
btrghstk@gmail.com

Abstract

Cold-start latency in large language model (LLM) serving is often reduced to a hardware question: how quickly can storage stream checkpoint bytes? This paper argues that the better abstraction is *workload shape*. A checkpoint layout determines the file and range requests exposed to the operating system; an I/O backend determines how those requests are submitted and completed; Python bindings and serving engines determine how much of any backend gain reaches the user. Consequently, the best loader is not a universal API choice, but a policy conditioned on format, scale, runtime capability, and integration layer.

We present Tensora, an open checkpoint-loading framework and measurement harness for evaluating that policy space. Tensora runs synchronous POSIX I/O, Tokio-based async I/O, Linux `io_uring`, and an adaptive default policy over two layouts: contiguous SafeTensors shards and a partitioned ServerlessLLM-style layout. The evaluation spans three layers—Rust backend attribution, Python bindings, and a custom vLLM integration—six public checkpoints from 0.6B to 32B parameters, and three environments: an H100 host with local NVMe and working `io_uring`, plus H200 and A100 container environments where `io_uring` is blocked.

The results show four regime changes. First, H100 SafeTensors has a scale crossover: sync is fastest at small scales, while `io_uring` is fastest for 8B–32B multi-shard checkpoints. Second, the ServerlessLLM-style layout changes the ranking: sync is never fastest in the H100 matrix, and async, `io_uring`, and default read planning trade places depending on partition structure. Third, integration layers reorder but preserve loader impact: Python materialization dominates absolute time, while vLLM cold `load_only` improves by 37–48% for 8B and 32B models with Tensora's adaptive SafeTensors path relative to vLLM's native loader. Fourth, ordering regimes are more portable than backend identities: small SafeTensors loads favor sync across all three environments, but the mid-to-large-scale winner changes with storage topology and backend availability.

The operational conclusion is deliberately bounded: use a format-aware adaptive loader with explicit overrides and capability checks. Do not choose sync, async, or `io_uring` as a universal default by name alone.

Keywords large language models · checkpoint loading · I/O policy · io_uring · cold start · vLLM · reproducibility

1 Introduction

An LLM serving request cannot generate a token until model weights are resident. In serverless and multi-tenant settings, idle models are commonly evicted to recover scarce accelerator memory; the next request then pays a cold load before inference begins. A 7B float16 checkpoint is already on the order of 14 GB, and the checkpoints in this paper reach beyond 65 GB of weight bytes. Cold loading is therefore not a startup footnote. It is part of the user-visible serving path.

The usual shorthand is to call this a bandwidth problem. That shorthand is incomplete. A loader does not read an abstract number of gigabytes; it executes a concrete read plan. The checkpoint format decides whether bytes appear as a few contiguous shards or as many indexed ranges across partition files. The backend decides whether those reads are submitted through blocking POSIX calls, an async runtime, or `io_uring`. The integration layer then adds its own costs: Python object construction, tensor materialization, CUDA context setup, allocator initialization, and serving-engine configuration. A device can have high peak throughput and still produce poor cold-start time when request geometry and submission policy are mismatched.

This paper studies a single thesis:

> **Checkpoint loading should be selected from workload shape.** The right backend depends on checkpoint layout, model scale, backend availability, and the layer at which loading is observed. A fixed rule such as "always sync," "always async," or "always `io_uring`" is the wrong abstraction.

The thesis is visible from two layouts. SafeTensors presents mostly whole-file or multi-shard eager loading: after metadata parsing, the loader must move large contiguous byte regions. ServerlessLLM-style partitioning presents a different workload: tensors are reconstructed from indexed partition files, creating many grouped range reads. These layouts are not cosmetic alternatives; they expose different I/O regimes. Scale then moves the boundary within a regime. A backend with low orchestration overhead can win for a small single-shard load, while a backend that sustains deeper submission queues can win for a large multi-shard load.

We built Tensora[1], an open framework for measuring these regimes. Tensora exposes synchronous POSIX I/O, Tokio-based async loading, Linux `io_uring`, and a feature-driven `default` policy under shared format abstractions. We evaluate six public checkpoints from 0.6B to 32B parameters, two layouts, and three observation layers: Rust backend attribution, Python bindings, and a custom vLLM integration. The main environment is an H100 host with local NVMe and working `io_uring`; H200 and A100 container environments with network-attached storage and blocked `io_uring` test whether the policy degrades gracefully when the strongest H100 backend is unavailable.

The data support a regime claim, not a universal-loader claim. On H100 SafeTensors, explicit sync wins at the small end of the ladder, while explicit `io_uring` wins from Qwen3-8B through Qwen3-32B. On H100 ServerlessLLM, sync is never fastest; async, `io_uring`, and default read planning lead different rows. Python and vLLM reorder backend rankings because they measure different constructs, but the effect survives integration: in vLLM cold `load_only` runs, Tensora's adaptive SafeTensors path reduces initialization by 37–48% relative to vLLM's native loader for 8B and 32B checkpoints. Cross-environment results preserve the most stable structural fact—small SafeTensors loads favor sync—while showing that the large-scale winner depends on kernel and storage context.

Contributions.

1. A workload-shape characterization of LLM checkpoint loading: format-level request geometry, not peak device bandwidth alone, constrains which backend policy is plausible.
2. A controlled Rust-layer measurement matrix comparing sync, async, `io_uring`, and adaptive default across SafeTensors and ServerlessLLM-style layouts.
3. Cross-layer evidence showing how Python bindings and vLLM integration reorder, but do not erase, backend effects.
4. Cross-environment validation across H100, H200, and A100 systems showing that ordering regimes are more portable than backend identities.
5. An operational policy recommendation: use a format-aware adaptive selector with explicit overrides, capability checks, and measured workload features instead of a universal I/O backend.

Roadmap. Section 2 positions the gap. Section 3 describes Tensora and the adaptive policy. Section 4 defines methodology. Section 5 presents results across three layers. Section 6 interprets implications, Section 7 bounds claims, and Section 8 concludes.

[1] Tensora is this paper's checkpoint-loading framework (Rust crate `tensora`, Python package `tensora_py`). It is unrelated to Google's TensorStore library for array storage.

2 Related Work

Tensora sits between four lines of work: LLM serving engines, serverless cold-start systems, checkpoint formats and loading infrastructure, and operating-system I/O policy. Prior work either optimizes the serving pipeline after weights are resident, avoids cold loads through placement and caching, changes the checkpoint/storage substrate, or improves kernel-level page service. Tensora targets the application-level loading decision: given a concrete checkpoint layout and host capability set, which userspace backend should execute the read plan?

2.1 LLM serving systems

Modern serving systems primarily optimize inference after weights are resident. vLLM introduces PagedAttention for KV-cache management and high-throughput serving [1]. Orca uses iteration-level scheduling [2]. Sarathi-Serve studies the prefill–decode scheduling trade-off [3]. DeepSpeed Inference applies model parallelism and kernel optimizations [4]. These systems make the steady-state serving pipeline efficient, but typically treat checkpoint loading as a fixed initialization cost.

Our vLLM integration is therefore a boundary test. Rust measurements explain backend mechanics; vLLM measurements test whether those mechanics survive in an engine with CUDA setup, memory pools, runtime configuration, and generation state. The contribution is not a new serving scheduler. It is a loader policy whose deployment impact can be observed inside a serving engine.

2.2 Serverless and cold-start mitigation

Serverless LLM systems make cold start a first-order concern because evicted models must be restored on demand. ServerlessLLM [5] proposes multi-tier loading across GPU, CPU memory, and storage. HydraServe [6] reduces cold-start latency in public-cloud serving through placement and pipeline parallelism. Snapshot-based warm-up systems [7] pre-materialize process state to bypass portions of startup.

These systems optimize *whether*, *where*, or *when* loading occurs. Tensora asks the complementary question that remains when a load is unavoidable: how should the required reads be submitted? The answer matters in any tiered design because each tier still presents a read plan to userspace.

2.3 Checkpoint formats and loading systems

Checkpoint formats define the request geometry visible to a loader. SafeTensors [8] stores tensor bytes in safe, contiguous files or shards. GGUF targets memory-mapped CPU inference and quantized deployments. ServerlessLLM's layout [9] distributes tensors across partition files with a central index, enabling tiered or partial placement at the cost of more range-oriented access.

Recent systems work attacks loading from other layers. BlitzScale [10] shares already-loaded models across inference instances over high-speed interconnects. PPC/MAIO [11] uses a programmable kernel page cache to adapt prefetching and eviction to LLM I/O patterns. MAIO operates below the system-call interface; Tensora operates above it. The two approaches are complementary: kernel cache policy changes how pages are served, while userspace backend policy changes how reads are issued.

A separate characterization line studies checkpoint/restore I/O patterns on parallel file systems [12], including buffered versus direct paths and liburing. That work focuses on training checkpointing in HPC environments. Tensora focuses on inference-time checkpoint loading in single-node serving and follows the effects through Python and vLLM.

2.4 I/O backend policy

The Linux I/O interface landscape includes blocking POSIX calls, async runtimes layered above kernel primitives, POSIX AIO, and `io_uring`. Database research provides the closest methodological warning: Jasny et al. [13] show that `io_uring` is not automatically faster. Its benefit depends on request shape, queue depth, batching, and implementation details. Bulk sequential reads can already amortize syscall overhead; fragmented or high-concurrency reads can benefit from batched submission.

LLM checkpoint loading spans both regimes. SafeTensors resembles contiguous bulk transfer with shard-level fanout. ServerlessLLM-style partitioning resembles grouped range access. Tensora translates the workload-shape lesson to LLM loading and adds three elements missing from prior I/O studies: format-conditioned

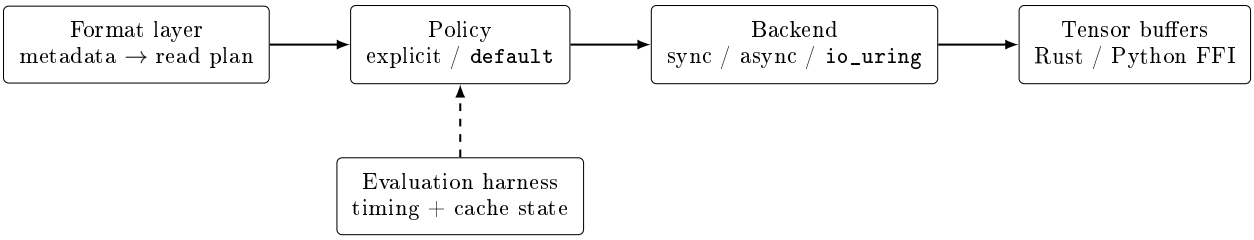

Figure 1: Tensora architecture. The hot path is format parsing → policy → backend execution → tensor buffers. The harness controls timing and cache state outside that path.

comparison, cross-layer measurement, and capability-gated fallback when `io_uring` is unavailable in containerized environments.

2.5 Positioning

We do not introduce a checkpoint format, a serving scheduler, or a kernel cache. We measure the load mechanics between them: four backend policies, two layouts, six public checkpoints, three measurement layers, and three deployment environments. The contribution is operational and falsifiable: backend choice depends jointly on layout, scale, and runtime capability; therefore a production loader should select from measured workload features and expose explicit overrides.

3 Tensora Design

Tensora is the measurement vehicle for this paper. Its design goal is not to hide loading behind a single convenient API, but to make the workload shape, backend choice, and integration overhead explicit enough to measure.

3.1 Design principles

Tensora follows four principles.

1. **One logical workload, many executions.** A format parser should produce the same tensor read plan regardless of backend, so timing differences reflect submission and planning policy rather than different work.
2. **Backend policy is explicit.** Sync, async, `io_uring`, and `default` are separate labels in the harness. The adaptive path is not allowed to disappear into an opaque fast path.
3. **Measurement stays outside the loader.** Cache drops, timing loops, environment capture, and export formatting are harness responsibilities. Loader code should load checkpoints, not manufacture benchmark conditions.
4. **The policy must cross layers.** A selector that only works in a Rust microbenchmark is incomplete. Tensora carries the same core into Python bindings and a vLLM loader integration.

3.2 Architecture

Tensora separates format parsing, backend execution, policy selection, and evaluation (Figure 1). The **format layer** parses checkpoint metadata and produces a request-local read plan: files, byte ranges, tensor names, and expected sizes. The **policy/backend layer** either executes an explicit backend selected by the caller or invokes the adaptive selector. The **evaluation layer** contains the Rust profiler, Python benchmarks, and vLLM experiments; it observes boundaries without being part of the production load path.

At the SafeTensors layer, the default load path is `Model::load(path)` → workload statistics → `choose_load_backend(stats)` → `load_sync`, `load_async`, or `load_io_uring`. The Linux implementation checks `backend_capabilities()` before selecting `io_uring`. ServerlessLLM follows the same structure after `compile_load_plan(directory)` builds an index-derived partition plan.

3.3 Common backend surface

All eager backends implement the same conceptual operations:

- `load(path)` for one full file,
- `load_batch(paths)` for multiple whole-file loads,
- `load_range(path, offset, len)` for one byte range, and
- `load_range_batch(requests)` for grouped range reads.

This surface is deliberately small. SafeTensors mostly exercises whole-file and batch loading; Serverless-LLM exercises range batches. Backend implementations may chunk, group, coalesce, or tune queue depth internally, but they must satisfy the same logical tensor plan.

3.4 Workload features

The selector and analysis use features available before reading tensor payloads:

- **Total bytes**: aggregate checkpoint payload size.
- **Shard or partition count**: number of files contributing to the load.
- **Average shard or partition size**: a coarse scale signal for fanout and queue-depth potential.
- **Request structure**: whole-file sequential reads versus indexed ranges.
- **Backend capability**: whether `io_uring` can be initialized on the host.

These features intentionally stop short of learning a per-file oracle. The policy should be explainable to an operator: small contiguous loads tend to favor low-overhead sync; large multi-shard loads can benefit from deeper submission; partitioned range loads need grouping and batching.

3.5 Adaptive loading policy

The `default` path is an adaptive policy plus read planning, not a synonym for one explicit backend. It first extracts workload features, then applies a capability-gated selector, then lets the chosen backend plan execution. On Linux, `choose_load_backend_with_capabilities(stats, capabilities)` prevents `io_uring` selection when ring creation is blocked or unsupported. This matters in the H200 and A100 environments, where container policy returns `EPERM`.

For SafeTensors, the selector uses total bytes, shard count, and average shard size to separate small contiguous loads from large multi-shard loads. For ServerlessLLM, it uses partition and range structure; the H100 matrix shows that a sync-first rule from SafeTensors should not be imported into this layout. The policy is therefore format-aware by construction.

The selector is intentionally not presented as an oracle. Section 5.4 reports where it wins, where it is near parity, and where it misses. Those misses motivate explicit overrides. A deployable adaptive loader should expose both: a good default for stable regimes and manual labels for known boundary cases.

```
format metadata → bytes, shards/partitions, request structure
                                  ↓
           capability gate:  is io_uring available?
                                  ↓
                select backend + execution plan
                                  ↓
                  materialize tensor buffers
```

Figure 2: Adaptive policy control flow. The selector uses workload features and backend availability before executing backend-specific planning.

3.6 Backend implementations

The **sync** backend uses blocking POSIX reads with practical host-side parallelism for large files. It is not a strawman serial baseline; its small-model wins are meaningful because it has low orchestration overhead.

The **async** backend is built on Tokio. It groups work per file, batches dynamically, and avoids unnecessary task fanout. It is weak for H100 SafeTensors, where whole-file eager loads do not justify async overhead, but strong for partitioned and network-storage cases.

The `io_uring` backend uses persistent rings, batched submissions, lazy initialization, dynamic ring sizing, and workload-dependent chunk and depth planning. Its value depends on enough queued work; the paper never claims that `io_uring` is intrinsically faster by API name alone.

Table 1: Key design choices in the evaluated `io_uring` backend.

Aspect	Design
Ring lifetime	Persistent rings with lazy initialization
Submission	Batched SQEs and explicit in-flight management
Planning	Dynamic ring sizing, chunk sizing, coalescing, and depth limits
Fallback	Capability checks exclude `io_uring` when initialization fails

The mmap-backed path remains useful for lazy selective access, especially through Python handles, but it is not the main eager cold-start mechanism evaluated in the Rust matrices. Eager loading and mmap answer different questions.

3.7 Formats and integrations

Tensora supports two layouts because they expose different regimes. `SafeTensors` presents whole-file or multi-shard contiguous loading. `ServerlessLLM` presents an indexed partition layout with range-oriented reads. Supporting both under one framework is what makes the layout-dependent policy claim measurable.

Python bindings are implemented with PyO3 and expose a small API: adaptive eager load, explicit sync and async eager load, and lazy `open_*` handles. The async Python path uses a shared Tokio runtime rather than constructing one per call. The vLLM integration uses a custom loader path and follows vLLM's synchronous loader boundary, so the serving benchmark reflects the interface available to real integrations rather than an invented async serving path.

4 Experimental Setup

The evaluation is designed to separate mechanism from deployment impact. A Rust benchmark isolates backend execution. A Python benchmark measures the public binding surface. A vLLM benchmark measures the serving-engine boundary. The same timing number should not be interpreted the same way across all three layers.

4.1 Measurement layers

1. **Rust backend attribution.** Measures checkpoint reading, format handling, and tensor assembly through a selected backend. It excludes Python interpreter costs and serving-engine GPU initialization. This layer answers which submission policy is fastest for a fixed logical read plan.
2. **Python public API.** Measures what a Python client observes: FFI transitions, tensor construction, object materialization, and binding overhead in addition to the backend path. This layer answers whether Rust-level differences survive the library interface.
3. **vLLM integration.** Measures loading inside a serving engine, including CUDA context setup, memory pools, runtime configuration, model initialization, and time-to-first-token when requested. This layer answers whether loader choice matters in an end-to-end serving context.

These layers are deliberately not collapsed. A Rust result provides backend evidence; a vLLM result provides deployment evidence. The limitations section states where claims cannot move across layers.

4.2 Research questions

The study answers five questions.

- **RQ1: Backend attribution.** How much does backend choice affect checkpoint loading for real LLM checkpoints?
- **RQ2: Layout dependence.** Does changing checkpoint layout change the backend ranking?
- **RQ3: API overhead.** How much does the Python binding layer add, and can binding design reduce it?
- **RQ4: Serving impact.** Do loader effects survive inside vLLM initialization and TTFT?
- **RQ5: Policy.** What should an operational default promise, and where should explicit overrides remain available?

RQ1–RQ2 use the Rust matrices, RQ3 uses Python benchmarks, RQ4 uses vLLM, and RQ5 combines all layers.

4.3 Platforms

Measurements were collected on three Linux environments (Table 2). H100 is the primary local-NVMe site. H200 and A100 are network-storage container environments where `io_uring` ring initialization fails with `EPERM`; they test fallback behavior rather than reproducing the exact H100 hardware path.

Table 2: Experimental platforms. H100 provides local NVMe and working `io_uring`; H200 and A100 provide restricted container environments with network-attached storage and blocked `io_uring`.

	H100	**H200**	**A100**
GPU	NVIDIA H100	NVIDIA H200	NVIDIA A100 80GB
CPU	x86_64 multi-core	Intel Xeon 8568Y+	AMD EPYC 7543
RAM	∼180 GB	∼2 TB	∼1 TB
Storage	Local NVMe	Network-attached	Network-attached
`io_uring`	Available	Blocked (EPERM)	Blocked (EPERM)
Kernel	6.x (Ubuntu 24.04)	6.8.0-83 (Ubuntu 22.04)	6.5.0-45 (Ubuntu 22.04)

The final software stack used Python 3.12 and Rust 1.94. The optional vLLM dependency is pinned through `bindings/python/uv.lock` (resolved `vllm` 0.11.0). Exports record kernel and package details so that absolute timings can be interpreted with the host context.

4.4 Backends, layouts, and model ladder

The Rust layer evaluates four eager-loading policies:

- **sync**: blocking POSIX reads with dynamic chunking and host-side parallelism,
- **async**: Tokio-based loading with dynamic batching and per-file grouping,
- **io_uring**: Linux `io-uring` with persistent rings and workload-driven planning,
- **default**: the adaptive selector and planner described in Section 3.

The two layouts are **SafeTensors**, a whole-file or multi-shard contiguous layout, and **ServerlessLLM-style partitioning**, an indexed layout accessed through grouped range reads. Memory-mapped APIs are exposed for lazy Python access but are not treated as eager cold-load backends in the Rust matrix.

The model ladder contains six public checkpoints: `Qwen/Qwen3-0.6B`, `HuggingFaceTB/SmolLM3-3B`, `Qwen/Qwen3-4B`, `Qwen/Qwen3-8B`, `Qwen/Qwen3-14B`, and `Qwen/Qwen3-32B`. This ladder spans small, medium, and large checkpoints and includes a non-Qwen model to reduce overfitting to one family. A single model size could justify only a constant backend choice; the ladder is what exposes crossovers.

4.5 Cache state and comparability

Cold-cache behavior is the primary condition for Rust and Python eager-load measurements. A cold-cache run performs:

1. `sync`,
2. `echo 3 > /proc/sys/vm/drop_caches`,
3. benchmark invocation.

Warm-cache conditions skip the cache drop and reuse page-cache state from prior reads. On restricted containers where direct cache dropping is unavailable, the repository-supported page-cache eviction path is recorded as a methodological deviation.

Each backend receives the same logical tensor read plan for a given model and layout. Differences in wall time come from submission strategy, parallelism, chunking, coalescing, and buffer management. If `default` loses on a row, that is evidence about the selector and planner under the same workload constraints, not a different task.

4.6 Baselines and reporting

The Rust matrices compare Tensora backends against one another because the purpose is backend attribution under identical format handling. The vLLM matrix includes a deployment-style baseline: `native` is vLLM's stock loading path, while loaders prefixed `ts_` use Tensora-backed integrations through the same harness.

Full matrices report representative cold-cache times for readability. Five-repetition anchor sets validate selected small, medium, and large regimes and are reported in Appendix B. We treat small single-cell gaps cautiously unless supported by repeated anchors or consistent ordering across rows. The main claims rely on large effect sizes and repeated regime patterns: sync versus `io_uring` at H100 SafeTensors scale, sync's failure on ServerlessLLM, and cross-environment changes in large-scale winners.

4.7 Falsification criteria

The central claim would be weakened if one fixed explicit backend dominated both layouts and all scales, if sync remained fastest for the largest H100 SafeTensors checkpoints, or if sync became the strongest backend across the ServerlessLLM matrix. It would be refined, not falsified, if another host showed the same regime changes at different model sizes.

The adaptive-policy claim would be weakened if `default` systematically lost to one explicit backend across the full matrix. The observed misses are narrower: medium-scale SafeTensors and selected partitioned rows. They motivate explicit overrides rather than a return to a global fixed backend.

5 Results

The evaluation proceeds in three layers: Rust backend attribution (Section 5.3), Python binding overhead (Section 5.6), and vLLM integration (Section 5.7). Tables present core evidence; figures visualize regime crossovers. Section 5.4 compares the adaptive default against the best per-row explicit backend to bound the selector's effectiveness.

5.1 Reading the Results

The section follows the same control flow as the system. Rust results isolate backend attribution. Oracle analysis bounds the adaptive default against the best explicit backend on each row. Cross-environment results test capability-gated fallback. Python and vLLM results then show how the same choices behave once library and serving overhead are included.

Table 3 is the visual thesis of the paper. SafeTensors begins in the sync regime and shifts with scale. ServerlessLLM is not sync-led. Restricted environments replace `io_uring` with async or default. vLLM preserves the small-versus-large structure while adding engine overhead. A single fixed backend cannot reproduce this map.

Table 3: Regime map: strongest observed policy by layout, environment, and scale. Color encodes the winning policy in the corresponding matrix. The key visual fact is non-dominance: every backend color appears, and the pattern changes with layout and environment.

Layer / layout	0.6B	4B	8B	32B
H100 SafeTensors	`sync`	`sync`	`io_uring`	`io_uring`
H100 ServerlessLLM	`io_uring`	`async`	`default`	`default`
H200 SafeTensors	`sync`	`sync`	`async`	`async`
H200 ServerlessLLM	`async`	`async`	`async`	`default`
A100 SafeTensors	`sync`	`async`	`async`	`default`
A100 ServerlessLLM	`default`	`default`	`default`	`async`
vLLM SafeTensors	`sync`	`default`	`default`	`io_uring`

Table 4: Effect-size summary for representative regime claims. Ratios are computed as slower time divided by faster time; larger values indicate stronger evidence for a regime distinction.

Claim	Evidence row	Effect size
Small SafeTensors favors low-overhead sync/default over `io_uring`	H100 Qwen3-0.6B	sync is 2.09× faster than `io_uring`; default is 1.66× faster than sync
Large SafeTensors needs deeper submission	H100 Qwen3-32B	`io_uring` is 1.28× faster than sync
Partitioning removes sync as winner	H100 ServerlessLLM Qwen3-32B	default is 1.44× faster than sync
`io_uring` availability changes large-scale winner	H200 SafeTensors Qwen3-32B	async is 3.41× faster than sync when `io_uring` is blocked
Serving impact survives integration	vLLM Qwen3-32B cold load_only	Tensora default is 48.1% faster than native

5.2 Headline findings

Four patterns emerge from the Rust and integration matrices. **First, SafeTensors exhibits a backend crossover.** Synchronous loading is the fastest explicit backend for model sizes up to 4B parameters; io_uring is the fastest explicit backend from 8B parameters upward. No single fixed backend tracks both regimes.

Second, the partitioned ServerlessLLM layout reverses the ranking. Synchronous loading is never the fastest explicit backend for any model size in this ladder. The relevant trade-off is between async loading (strong on smaller and mid-sized range loads) and io_uring (competitive at larger partition counts).

Third, the adaptive default is useful but bounded. It beats the best explicit backend in 3 of 12 H100 Rust cells (Table 7) and lands within roughly 3% on several additional rows. The large misses are concentrated in medium-scale boundary regions. That pattern argues for adaptive defaults plus explicit overrides, not for hiding the selector or claiming oracle optimality.

Fourth, integration overhead reorders but does not erase backend effects. Python materialization makes absolute times much larger than Rust timings. vLLM adds engine initialization. Yet loader choice still changes cold initialization by tens of seconds at larger scales, which is the deployment-relevant outcome.

Fifth, ordering regimes transfer better than backend names. Cross-environment validation on H200 and A100 containers (where `io_uring` is blocked by kernel security policy) confirms that small SafeTensors loads favor sync across all environments. The mid-to-large-scale winner changes with storage topology and kernel capability. This makes capability-gated adaptation more valuable, not less.

5.3 Rust Backend Results

The Rust harness is the authoritative backend-attribution layer because it measures checkpoint loading directly, without Python tensor conversion or serving-engine initialization costs. Unless stated otherwise, each matrix cell is one representative cold-cache run; Appendix B reports five-repetition medians and min–max bands for selected anchors.

5.3.1 SafeTensors Results

Table 5 summarises the explicit backend comparison for SafeTensors.

Table 5: Rust cold-cache SafeTensors results (ms). The crossover from sync-favored small models to io_uring-favored large multi-shard models is the evidential core behind the adaptive policy.

Model	Sync	Async	Default	io_uring
Qwen3-0.6B	555.28	1416.19	333.59	1159.24
SmolLM3-3B	1331.79	5802.05	3722.35	3812.35
Qwen3-4B	1820.85	5039.21	3083.18	2967.42
Qwen3-8B	3880.40	7074.40	3844.35	3172.37
Qwen3-14B	7030.25	13489.00	6138.70	5977.07
Qwen3-32B	15454.45	28763.46	12119.50	12089.05

Two regimes are visible. Sync dominates at the small end: 555 ms for Qwen3-0.6B versus 1159 ms for io_uring. The ranking flips at Qwen3-8B: io_uring at 3172 ms versus sync at 3880 ms, a 1.22× advantage. At the largest scale (Qwen3-32B), io_uring is 1.28× faster than sync. The crossover is not a single threshold; it is a progression visible across the entire ladder.

Async is consistently the weakest backend for SafeTensors. Whole-file and multi-shard loads benefit from straightforward bulk parallelism rather than scheduler-mediated async orchestration, which adds overhead without corresponding I/O benefits on this workload shape.

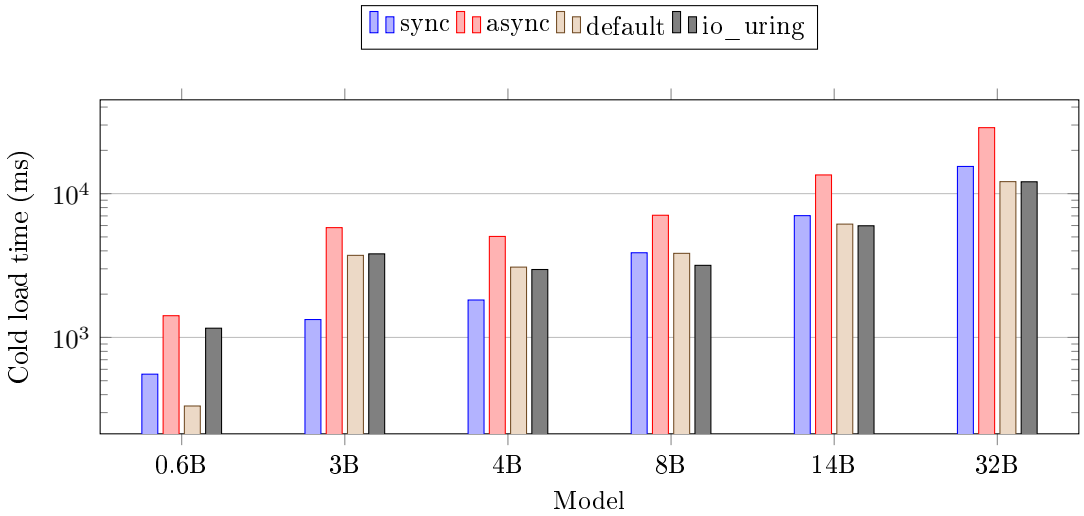

Figure 3: SafeTensors backend crossover (log scale). Sync dominates the left (small models); io_uring dominates the right (large multi-shard models). Default trails on medium-scale rows where the heuristic's feature mapping is imprecise.

Figure 3 visualizes the progression. `default` adds planning (chunking, coalescing) that helps on Qwen3-0.6B (333 ms, beating all explicit backends) but misses the medium-scale crossover region on SmolLM3-3B and Qwen3-4B.

5.3.2 ServerlessLLM Results

The ServerlessLLM layout produces a different ordering, shown in Table 6.

Sync is the slowest explicit backend on every ServerlessLLM row, between 1.1× (Qwen3-0.6B) and 1.4× (Qwen3-32B) behind the strongest alternative. The best explicit backend varies with scale: async dominates at smaller partition counts (SmolLM3-3B, Qwen3-4B), while io_uring becomes competitive at larger scales (Qwen3-14B, Qwen3-32B).

Table 6: Rust cold-cache ServerlessLLM results (ms). Synchronous loading never wins; the ranking shifts from async at smaller partition scales toward io_uring at larger sizes.

Model	Sync	Async	Default	io_uring
Qwen3-0.6B	1645.06	1471.34	1432.25	1387.57
SmolLM3-3B	3941.88	2391.24	3182.28	3091.49
Qwen3-4B	5137.72	3504.25	3551.82	3727.14
Qwen3-8B	11048.40	9463.37	7589.47	9043.51
Qwen3-14B	18661.25	16139.08	13422.94	13257.23
Qwen3-32B	39822.09	29180.89	27659.78	29779.86

`default` outperforms all explicit backends on Qwen3-8B (7589 ms vs 9044 ms for the next best, io_uring) and Qwen3-32B (27660 ms vs 29181 ms for async) because its read planning—coalescing nearby ranges, dynamic chunk sizing—benefits partitioned workloads more than any single explicit submission policy. It trails explicit async on SmolLM3-3B (3182 ms vs 2391 ms, a 1.33× gap), the same medium-scale fragility seen in SafeTensors.

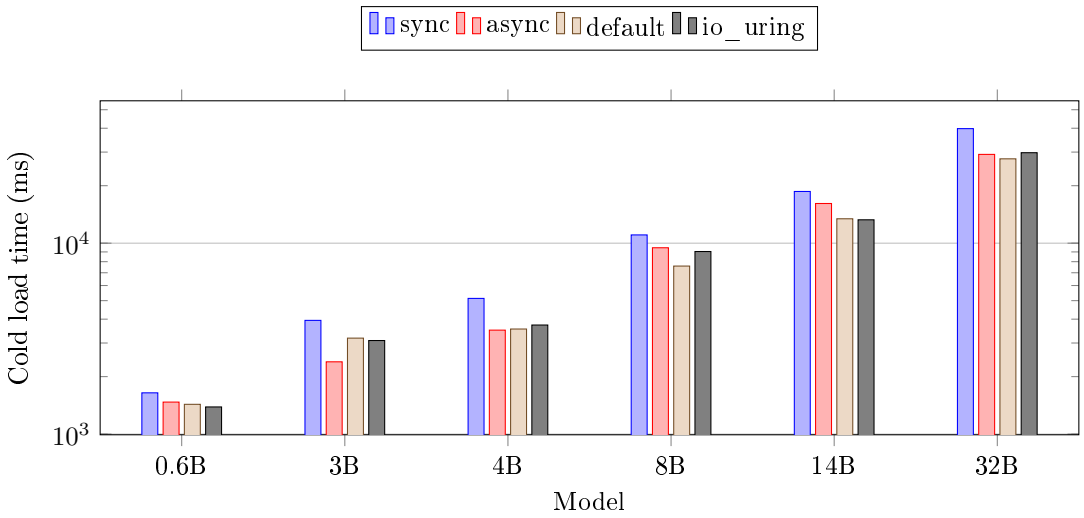

Figure 4: ServerlessLLM backend ordering (log scale). Sync is never the winner; the layout shifts the ranking toward async and io_uring. Default benefits from read planning on large partitioned loads.

Figure 4 shows the ranking shift. The figure confirms that the ServerlessLLM workload shape—many small range reads across partition files—rewards different backend policies than the SafeTensors contiguous regime.

5.4 Oracle analysis: how well does the adaptive default track the best backend?

The adaptive default is not expected to match the best per-row explicit backend in every cell—it is a heuristic, not an oracle. Table 7 quantifies the gap.

Table 7: Default versus per-row best explicit backend. Cells show `default` / `best_explicit` ratio. Ratios < 1.0 mean default wins; ratios > 1.0 mean default trails.

Model	SafeTensors	ServerlessLLM
Qwen3-0.6B	0.60	1.03
SmolLM3-3B	2.79	1.33
Qwen3-4B	1.69	1.01
Qwen3-8B	1.21	0.80
Qwen3-14B	1.03	1.01
Qwen3-32B	1.00	0.93

Default beats the best explicit backend in 3 of 12 cells and lands within roughly 3% on four additional cells. For SafeTensors, default correctly routes the smallest model (Qwen3-0.6B) to a fast path and tracks the large-scale io_uring regime within 3% at Qwen3-14B and near parity at Qwen3-32B. The medium-scale misses—SmolLM3-3B (2.79×), Qwen3-4B (1.69×), and Qwen3-8B (1.21×)—are the region where the crossover from sync to io_uring is most sensitive to per-checkpoint shard structure that the current feature set does not fully resolve.

For ServerlessLLM, default beats the best explicit backend in 2 of 6 cells and is close on most remaining rows. The single notable miss is SmolLM3-3B (1.33×), where async's grouped range execution fits the partition structure better than default's chosen path. The remaining ratios are near-parity (≤ 1.03).

Interpretation. The misses are not hidden; they are the point at which the adaptive policy becomes an engineering policy rather than an oracle. They identify where the feature mapping is fragile and where explicit overrides (for example, force sync for SafeTensors SmolLM3-3B; force async for ServerlessLLM SmolLM3-3B) provide clear value. A deployable selector should therefore expose overrides instead of pretending every boundary can be learned from one static threshold.

5.5 Cross-Environment Validation

The H100 results above were collected with all four backends available, including `io_uring`. Two additional environments—an H200 container and an A100 container, both with network-attached storage—block `io_uring` at the kernel level (`EPERM`). This constraint tests two questions: (1) whether the adaptive selector degrades gracefully when its strongest large-scale backend is unavailable, and (2) whether backend ordering regimes transfer across hardware.

5.5.1 H200 Results (io_uring blocked)

Table 8 shows the H200 SafeTensors matrix. With `io_uring` blocked, async replaces it as the fastest backend at large scale, while sync remains dominant for small models.

Table 8: H200 cold-cache SafeTensors results (ms). `io_uring` is blocked (`EPERM`). Sync dominates at small scale; async takes over at large scale in the absence of `io_uring`.

Model	Sync	Async	Default
Qwen3-0.6B	604.34	852.17	649.23
SmolLM3-3B	2013.66	2938.30	2130.51
Qwen3-4B	909.83	2478.10	941.24
Qwen3-8B	3924.69	2977.59	3213.01
Qwen3-14B	25872.03	4767.67	4906.13
Qwen3-32B	36205.91	10623.83	12356.23

The H200 SafeTensors crossover preserves the structural pattern from H100: sync wins at small scale (0.6B–4B), while a different backend dominates at large scale (8B+). On H100 that backend is `io_uring`; on H200 it is async. The selector adapts: `default` tracks within 3–8% of the best available backend for 0.6B–14B, though it trails async by 16% at 32B.

Table 9 shows the H200 ServerlessLLM matrix.

Table 9: H200 cold-cache ServerlessLLM results (ms). Async dominates most rows; `default` beats both explicit backends at 32B.

Model	Sync	Async	Default
Qwen3-0.6B	1565.27	1491.44	1586.84
SmolLM3-3B	3945.68	1947.39	1987.07
Qwen3-4B	5100.25	2489.27	2809.79
Qwen3-8B	10788.88	6045.47	6098.06
Qwen3-14B	17765.28	14450.40	15453.47
Qwen3-32B	39061.57	27367.74	24259.15

The ServerlessLLM ordering on H200 matches H100 structurally: sync is never the fastest, and async dominates most rows. `default` outperforms both explicit backends at 32B (24259 ms vs 27368 ms for async), consistent with the H100 finding that read planning benefits partitioned loads at large scale.

5.5.2 A100 Results (io_uring blocked, reduced ladder)

The A100 environment runs a four-model reduced ladder (0.6B, 4B, 8B, 32B) with the same `EPERM` constraint. Table 10 and Table 11 summarise the results.

Table 10: A100 cold-cache SafeTensors results (ms). Network-attached storage with AMD EPYC CPU. Sync wins at 0.6B; async dominates at 4B–8B; sync recovers at 32B.

Model	Sync	Async	Default
Qwen3-0.6B	724.26	1094.43	965.95
Qwen3-4B	7598.41	3532.11	7810.53
Qwen3-8B	8412.36	4235.21	11896.57
Qwen3-32B	46923.42	60807.34	43816.68

Table 11: A100 cold-cache ServerlessLLM results (ms). `default` wins at 0.6B, 4B, and 8B; async dominates at 32B.

Model	Sync	Async	Default
Qwen3-0.6B	2183.33	2250.91	1554.97
Qwen3-4B	7551.79	4486.75	3523.25
Qwen3-8B	14707.01	11528.26	11431.61
Qwen3-32B	56025.98	40472.54	44854.60

The A100 SafeTensors results show a distinctive pattern: sync wins at 0.6B, async dominates the mid-range (4B–8B), but sync recovers at 32B. This contrasts with both H100 (where `io_uring` dominates large scale) and H200 (where async dominates large scale). The `default` selector on A100 performs well at 0.6B and 32B but misses the mid-range async regime, producing its largest gap at 8B (2.81× behind async). For ServerlessLLM, `default` wins at 0.6B and 4B and tracks within 1% of async at 8B.

5.5.3 Cross-Environment Summary

Table 12 summarises the best explicit backend per row across all three environments.

Table 12: Best explicit backend by environment for SafeTensors. The crossover from sync to another backend persists on H100 and H200; A100 shows a non-monotonic pattern where sync recovers at 32B. Which alternative wins at mid-to-large scale depends on hardware and storage topology.

Model	H100	H200	A100
Qwen3-0.6B	sync	sync	sync
Qwen3-4B	sync	sync	async
Qwen3-8B	io_uring	async	async
Qwen3-32B	io_uring	async	sync

Three findings from the cross-environment comparison: (1) Sync dominates small SafeTensors models in all three environments—this is the most portable ordering. (2) At large scale, the strongest backend depends on available kernel interfaces and storage topology: `io_uring` on local NVMe (H100), async on network storage (H200), and sync on a different network-storage configuration (A100 at 32B). (3) The adaptive selector degrades gracefully: when `io_uring` is unavailable, it produces within-regime-competitive results on most rows, confirming that the capability-gated fallback mechanism works as designed.

5.6 Python Results

Python benchmarks were collected on Qwen/Qwen3-8B to quantify user-facing overhead and to evaluate whether binding optimisation improved practical performance.

The most important Python result is that binding engineering matters. Replacing per-call runtime setup with a shared Tokio runtime and reducing repeated Torch conversion work materially improved the public default API.

Table 13: Python eager-load times for Qwen3-8B (seconds). Binding tuning and selector redesign both contributed measurable improvements.

Case	Baseline bindings	After binding tuning	After selector tuning
SafeTensors default	15.88	14.55	11.55
ServerlessLLM default	22.45	20.50	18.45

Table 13 shows the sequence. For SafeTensors on Qwen3-8B, the default path moves from 15.88 s to 11.55 s, a 27.3% reduction from the baseline. For ServerlessLLM, the same sequence moves default from 22.45 s to 18.45 s, a 17.8% reduction.

The Python numbers are much larger than the Rust-only numbers (11.55 s vs 3.84 s for SafeTensors default) because eager loading at the binding layer includes substantial tensor materialization and Python object construction. The key conclusion: Python is not a transparent pass-through layer. Backend improvements matter, but they are filtered through the cost of converting checkpoint contents into Torch tensors and Python dictionaries.

5.7 vLLM Results

The final vLLM matrix completed successfully across all six models, all benchmark loaders, warm and cold cache conditions, and all three benchmark kinds.

Stock vLLM versus Tensora. The `native` loader follows vLLM's default weight-loading path; `ts_*` loaders use Tensora-backed integrations. On cold `load_only`, Tensora's default SafeTensors path reduces initialization time relative to `native` across all measured scales (Table 14).

Table 14: Cold vLLM load_only: native vs. Tensora (ms). The gap is an end-to-end effect (CUDA context, memory pools, and loader policy), not a pure Rust microbenchmark.

Model	`native`	`ts_safetensors_default`	Δ
Qwen3-0.6B	23659	20578	−13.0%
Qwen3-8B	56639	35556	−37.2%
Qwen3-32B	169971	88200	−48.1%

The improvement grows with model size: 13% for the smallest model, 37% at 8B, and 48% at 32B. This is consistent with the Rust SafeTensors finding (io_uring advantage grows at larger scales) filtered through vLLM's fixed initialization overhead.

For cold load_only across all models, the best loader varies with model size. Table 15 shows the strongest loader in the final cold-load-only configuration.

Table 15: Best cold load_only vLLM loader by model. The pattern follows the Rust SafeTensors regime: sync-style loading at small scales, io_uring at large scales.

Model	Best init time (ms)	Best loader
Qwen3-0.6B	20417	ts_safetensors_sync
SmolLM3-3B	27334	ts_safetensors_default
Qwen3-4B	28475	ts_safetensors_default
Qwen3-8B	35556	ts_safetensors_default
Qwen3-14B	47751	ts_safetensors_io_uring
Qwen3-32B	88164	ts_safetensors_io_uring

The vLLM best-loader pattern mirrors the Rust SafeTensors crossover: smaller models favor sync-style loaders, larger models favor default or io_uring. The match is not exact—vLLM initialization includes fixed costs that dilute the pure-I/O ranking—but the regime structure survives.

For TTFT (Table 16), the ranking is broadly similar to load_only but does not match row-for-row because TTFT includes the first generation step after initialization.

Table 16: Best cold TTFT vLLM loader by model.

Model	Best TTFT (ms)	Best loader
Qwen3-0.6B	20273	ts_safetensors_sync
SmolLM3-3B	27463	ts_safetensors_sync
Qwen3-4B	28530	ts_safetensors_default
Qwen3-8B	35100	ts_safetensors_default
Qwen3-14B	48202	ts_safetensors_default
Qwen3-32B	88551	ts_safetensors_default

The SafeTensors loader family consistently outperforms the ServerlessLLM family for both metrics. Load_only and TTFT show backend effects mainly in initialization; decode is comparatively insensitive once weights are resident.

5.8 Layer summary

Four principles emerge. (1) Backend ranking depends on workload shape: SafeTensors favors sync at small scale and `io_uring` at large H100 scale, while ServerlessLLM favors async, `io_uring`, or default planning and disfavors sync. (2) The adaptive default is useful but bounded: it wins or ties several stable-regime rows and misses medium-scale rows that need explicit overrides. (3) Integration layers reorder but do not eliminate backend differences: Python adds tensor conversion overhead and vLLM adds initialization cost, yet loader choice still changes user-visible startup. (4) Cross-environment validation confirms that sync dominates small SafeTensors models everywhere; the mid-to-large-scale winner varies with kernel interface availability and storage topology, and A100 shows a non-monotonic pattern where sync/default recovers at the largest scale. The selector's capability-gated fallback adapts to these constraints better than a hard-coded `io_uring`-first policy would.

6 Discussion

The main result is not that one backend wins. The main result is that a checkpoint loader must treat layout and scale as policy inputs.

6.1 Mechanism: why layout changes the answer

SafeTensors and ServerlessLLM-style partitioning create different I/O problems. SafeTensors exposes large contiguous reads after metadata parsing. For small checkpoints, a practical sync backend has low overhead and enough parallelism; additional scheduling machinery does not amortize. As size and shard count grow, the H100 workload has enough in-flight work for `io_uring` to pay off.

ServerlessLLM-style partitioning changes the request geometry. The loader reconstructs tensors from indexed partition files, so grouping, coalescing, and range batching dominate. The SafeTensors rule "sync can be good for small loads" does not transfer: sync never wins in the H100 ServerlessLLM matrix. The format, not the API name, explains the reversal.

6.2 Integration: why rankings move across layers

Rust, Python, and vLLM answer different questions. Rust isolates backend mechanics. Python measures a public library boundary with FFI, tensor conversion, and object construction. vLLM measures serving startup with CUDA context setup, memory pools, runtime configuration, and generation setup. These costs can dilute a pure I/O advantage or reorder two close backends.

This is not a flaw in the methodology; it is the deployment reality. A systems paper about loading should show both mechanism and propagation. Tensora's gains matter because they remain visible in vLLM cold initialization even after fixed engine costs are included.

6.3 Policy: what `default` should promise

An adaptive default should promise three bounded properties. First, it should avoid clearly wrong global choices: sync for all partitioned loads, or `io_uring` for every tiny contiguous load. Second, it should degrade gracefully when a backend is unavailable, as in containers that block `io_uring`. Third, it should expose explicit overrides for known boundary cases.

It should not promise oracle optimality. Table 7 shows misses where the current feature set is too coarse. Those misses are useful because they identify where the next policy should add signals: finer shard geometry, storage topology, or learned per-environment calibration. The correct engineering response is transparency plus overrides, not hiding the selector behind a single API name.

6.4 What transfers

Absolute milliseconds are machine-specific. Ordering regimes transfer more plausibly. In this study, small SafeTensors loads favor sync across H100, H200, and A100. The large-scale winner does not transfer by name: H100 favors `io_uring`, H200 favors async when `io_uring` is blocked, and A100 shows a non-monotonic recovery at the largest SafeTensors row. A replication should therefore ask whether workload-shape changes persist, not whether every number matches.

6.5 Implications

For practitioners, log the features that determine loader choice: format, bytes, shard or partition count, backend availability, cache condition, and integration layer. Expose a default, but also expose explicit backend labels and override hooks.

For researchers, report loading results as layered constructs. A Rust microbenchmark, a Python API benchmark, and a serving-engine startup benchmark are all valuable; none subsumes the others. A paper that reports only one layer risks explaining the wrong phenomenon.

7 Limitations

The paper makes a bounded systems claim: backend policy should depend on workload shape. It does not prove a universal selector, universal thresholds, or universal superiority of any I/O API.

7.1 External validity

The primary matrices come from one H100 host with local NVMe and working `io_uring`. H200 and A100 provide useful but different validation: network-storage containers where `io_uring` is blocked. They test fallback behavior and regime transfer, but they are not three identical replicas of the same hardware/storage stack.

The model ladder covers six public checkpoints from 0.6B to 32B parameters and two layouts. It does not cover GGUF, quantized-only deployments, object stores, distributed filesystems, RAID variants, direct-I/O configurations, Windows/macOS backends, or multi-node distributed loading.

7.2 Construct validity

"Load time" means different things at different layers. Rust timings approximate backend-attributed disk-to-tensor work. Python timings add interpreter and binding costs. vLLM timings add engine initialization, CUDA setup, memory pools, and generation state. A Rust result should not be quoted as a serving result, and a vLLM result should not be used to rank pure I/O APIs without qualification.

7.3 Statistical limits

Most full-matrix cells are representative cold-cache runs. Anchor repetitions validate selected small, medium, and large regimes, but not every row has repeated statistics. The paper therefore emphasizes large effect sizes, ordering reversals, and repeated structural patterns rather than treating every small difference as significant. Differences below roughly 10% should be read cautiously unless supported by anchors or cross-row consistency.

7.4 Selector boundedness

The adaptive default is not an oracle. On H100 SafeTensors it trails explicit sync by 2.79× for SmolLM3-3B and 1.69× for Qwen3-4B, and trails explicit `io_uring` by 1.21× for Qwen3-8B. On H100 ServerlessLLM it trails explicit async by 1.33× for SmolLM3-3B. On A100, it misses the mid-range async regime for SafeTensors.

These failures are part of the result. They show that a deployable policy needs explicit overrides and likely per-environment calibration. They do not rescue a fixed global backend: no explicit backend dominates across layout, scale, and environment.

7.5 Future work

The next step is not to claim a perfect heuristic. It is to expand calibration: more local-NVMe hosts, filesystem and mount-option sweeps, direct-I/O variants, object-store backends, additional checkpoint formats, and learned or profile-guided selectors. The current evidence justifies the abstraction—workload-shaped loading—and identifies where a stronger selector should improve.

8 Conclusion

Checkpoint loading for LLM serving is not a single bandwidth number. It is a workload-shaped systems problem. The layout determines request geometry, the backend determines submission policy, runtime capability determines which interfaces can be used, and the integration layer determines how much of a backend improvement reaches users.

Tensora measures this path across two layouts, four backend policies, three observation layers, six public checkpoints, and three environments. The results reject a universal backend. H100 SafeTensors shifts from sync-favored small loads to `io_uring`-favored large loads. H100 ServerlessLLM-style partitioning eliminates sync as a winner and rewards grouped range execution and planning. Python and vLLM reorder the details but preserve the practical consequence: loader choice changes public API time and serving initialization. H200 and A100 show that regime structure transfers better than backend identity.

The recommendation is therefore simple and bounded: load by design. Select from measured workload features, gate the choice by runtime capability, and expose explicit overrides for boundary cases. The right abstraction is not an I/O API name; it is a format-aware adaptive policy.

A Reproducibility

This appendix describes how to rebuild the paper and reproduce the ordering evidence. The goal is traceability and comparable regimes, not identical milliseconds on different hardware.

A.1 Paper build

Build from `paper/`:

```
pdflatex -interaction=nonstopmode main.tex
bibtex main
pdflatex -interaction=nonstopmode main.tex
pdflatex -interaction=nonstopmode main.tex
```

The public source bundle should include `main.tex`, `template.tex`, `PRIMEarxiv.sty`, `sections/`, `figures/`, `library.bib`, and `00README.json`. If a venue template replaces the arXiv style, rebuild with that venue's class and rules.

A.2 Repository and revision

The implementation is open source at `https://github.com/botirk38/tensora`. Record the exact revision next to exported results:

```
git rev-parse HEAD
```

```
git status -sb
```

Also record `date -u`, `uname -a`, `rustc -version`, `python3 -version`, GPU and driver information from `nvidia-smi`, and the active vLLM package version when vLLM is used.

A.3 Rust profiling

Build the profiler in release mode:

```
cargo build --release --bin profile
```

Run one cell with:

```
cargo run --release --bin profile -- <format> <backend> \
  --model-id <MODEL_ID> --iterations 1
```

where `<format>` is `safetensors` or `serverlessllm`, and `<backend>` is `sync`, `async`, `io-uring`, or `default`. SafeTensors uses the Hugging Face Hub cache. ServerlessLLM-style conversions are stored under the Tensora cache directory, for example `$XDG_CACHE_HOME/tensora/<slug>/serverlessllm`.

For cold-cache runs on hosts with sufficient privileges, run `sync` followed by `echo 3 > /proc/sys/vm/drop_caches` before each invocation. On restricted containers, use the repository-supported eviction path and report that deviation. The full H100 matrix in `results/h100/rust.tsv` was generated by sweeping model ids, formats, and backends with that ritual.

A.4 Python and vLLM harnesses

Python benchmarks live under `bindings/python/benchmarks/`. The convenience script `scripts/run\_benchmarks.sh` runs pytest-benchmark suites and writes JSON outputs. Set `TENSORA_BENCH_MODELS` to one or more Hugging Face model ids, separated by spaces or commas. Set `TENSORA_BENCH_NO_VLLM=1` when a GPU-backed vLLM environment is unavailable.

When vLLM is enabled, runs are serialized by model to avoid GPU memory interference. The paper's archived exports live under `results/h100/`, `results/h200/`, and `results/a100/`. The H100 vLLM matrix in `results/h100/vllm.tsv` sweeps loaders, cache modes, and benchmark kinds: `load_only`, `TTFT`, and `steady_state_decode`.

A.5 What a replication should check

A strong replication should check ordering regimes before exact milliseconds:

- SafeTensors changes backend preference with scale.
- ServerlessLLM-style partitioning does not follow the SafeTensors sync-first rule.
- Python overhead is large enough to change observed rankings.
- vLLM initialization is sensitive to loader choice, while steady-state decode is much less sensitive.
- Blocking `io_uring` changes large-scale winners and exercises fallback behavior.

B Cold-cache anchor statistics

Table 17 summarizes five cold-cache repetitions for selected H100 anchors from `results/h100/anchor.tsv`. These anchors sanity-check variance; the main matrices use one representative cold run per cell for readability.

C Artifact layout

Each results directory contains tab-separated summaries and raw exports. `results/<env>/rust.tsv` contains Rust cold-cache profiles. `results/<env>/vllm.tsv` contains vLLM integration benchmarks where available. `results/<env>/anchor.tsv` contains repeated anchors where collected. `results/<env>/raw/` contains pytest-benchmark JSON exports.

Table 17: Anchor repetitions: median cold load time (ms) with min–max across five runs.

Model	Format	Backend	Median	Min–Max
Qwen3-0.6B	SafeTensors	sync	323.35	322.05–402.93
Qwen3-0.6B	SafeTensors	default	324.86	321.04–495.10
Qwen3-8B	SafeTensors	sync	3700.68	3647.97–3731.11
Qwen3-8B	SafeTensors	io_uring	3250.29	3160.46–3274.00
Qwen3-32B	SafeTensors	sync	15462.39	15326.77–16143.61
Qwen3-32B	SafeTensors	io_uring	12204.11	11736.60–14388.76
Qwen3-8B	ServerlessLLM	io_uring	7267.02	7118.83–8884.83
Qwen3-8B	ServerlessLLM	default	8059.26	7586.28–9143.26
Qwen3-32B	ServerlessLLM	async	29010.56	28639.44–29222.41
Qwen3-32B	ServerlessLLM	io_uring	27910.41	26663.65–29152.43

References

[1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP 2023)*. ACM, 2023.

[2] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538. USENIX Association, 2022.

[3] Amey Agrawal, Nitin Kedia, Ashish Panwar, et al. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*. USENIX Association, 2024.

[4] Reza Yazdani Aminabadi et al. DeepSpeed inference: Enabling efficient inference of transformer models at unprecedented scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC 2022)*. IEEE/ACM, 2022.

[5] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. Serverlessllm: Low-latency serverless inference for large language models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*. USENIX Association, 2024.

[6] Anonymous Authors. HydraServe: Minimizing cold start latency for serverless LLM serving in public clouds. arXiv preprint arXiv:2502.15524, 2025.

[7] Paulo Silva and Thiago Emmanuel Pereira. Performance evaluation of snapshot methods to warm the serverless cold start, 2021.

[8] Hugging Face. Safetensors documentation. `https://huggingface.co/docs/safetensors`, 2024. Accessed: April 2026.

[9] ServerlessLLM Contributors. Serverlessllm. `https://github.com/ServerlessLLM/ServerlessLLM`, 2024. GitHub repository.

[10] Zizhao Mo et al. Blitzscale: Efficient large model serving with elastic GPU resource sharing. In *Proceedings of the 2024 USENIX Annual Technical Conference (ATC 2024)*. USENIX Association, 2024.

[11] Yubo Liu, Hongbo Li, Xiaojia Huang, Yongfeng Wang, Hanjun Guo, Hui Chen, Yuxin Ren, and Ning Jia. Accelerating model loading in LLM inference by programmable page cache. In *Proceedings of the 24th USENIX Conference on File and Storage Technologies (FAST 2026)*. USENIX Association, 2026.

[12] Nicolas Pretet et al. Understanding LLM checkpoint/restore I/O strategies and patterns, 2025.

[13] Matthias Jasny, Muhammad El-Hindi, Tobias Ziegler, Viktor Leis, and Carsten Binnig. High-performance DBMSs with io_uring: When and how to use it, 2025.